ICT3222 ADVANCED DATABASE MANAGEMENT SYSTEMS | UNIVERSITY COURSEWORK

Supermarket Chain Management System

Equitable 5-Member Work Allocation & ADBMS Evaluation Rubric Mapping

MYSQL 8.0 INNODB | SPRING BOOT 3 (PURE JDBC) | ANGULAR 17+ MATERIAL | 50,000+ ORDERS DATASET | STRICTLY NO ORM / JPA

1. Quick Scorecard: The 5-Member Database Matrix

Member	Focus Domain	Stored Procedures	Triggers & Functions	Views & Events	Viva Highlight
Member 1	**Sales & POS**	`sp_place_order` `sp_cancel_order`	`trg_orderitem_check_stock` `trg_orderitem_reduce_stock` `fn_order_total`	Composite PK clustering on `order_item`	**Pessimistic locking** (`FOR UPDATE`) & rollback
Member 2	**Inventory & Supply**	`sp_add_stock` `sp_receive_purchase` `sp_transfer_stock`	`trg_inventory_low_stock` `fn_stock_level`	`v_low_stock` `ev_weekly_low_stock_scan`	**Optimistic locking** (`version`) & inter-branch transfer
Member 3	**Catalog & Promotions**	`sp_save_product` `sp_register_customer` `sp_apply_discounts`	`trg_product_price_audit` `trg_rule_check` `fn_customer_tier` , `fn_coupon_valid`	`v_active_discounts` `ev_expire_discount_rules`	**Cursor discount engine** & `FULLTEXT` search
Member 4	**Analytics & Performance**	`sp_monthly_report` `sp_category_sales_cte`	`trg_daily_sales_log` `fn_avg_daily_sales`	`v_daily_sales` , `v_product_ranking` `ev_daily_sales_summary` `ev_monthly_log_purge`	**Window functions** (`RANK`), Partitioning & 50k `EXPLAIN`
Member 5	**Security, RBAC & Admin**	`sp_get_login` `sp_deactivate_employee` `sp_change_user_role`	`trg_employee_audit` `trg_security_alert` `fn_mask_phone` `fn_check_password_strength`	`v_masked_customer_dir` `v_security_audit_log`	**MySQL 8 Roles** (`CREATE ROLE`), Data Masking & PITR Backup

2. Detailed Member Work Breakdown

Member 1: Sales, Transactions & Point of Sale (POS)

Theme: High-concurrency transaction processing, locking, and atomic order placement

Database Tables Owned:

`sales_order`, `order_item`, `payment`

Database Programming Objects:

- **Procedures:** `sp_place_order` (JSON cart items via `JSON_TABLE`, `START TRANSACTION`, rollback on exception), `sp_cancel_order` (restores inventory & marks cancelled).
- **Triggers:** `trg_orderitem_check_stock` (BEFORE INSERT, locks row with `FOR UPDATE`), `trg_orderitem_reduce_stock` (AFTER INSERT, logs SALE).
- **Function:** `fn_order_total(order_id)`.
- **Clustering Proof:** Composite PK on `order_item(order_id, product_id)`.

Application Implementation:

- **Spring Boot:** `lk.supermarket.sales` (OrderController, OrderService, OrderDao, PaymentDao, deadlock retry logic).
- **Angular 17+:** `features/pos/` (Barcode/product search, cart state, discount breakdown, receipt view).

Extra Features for Top Marks:

- Two-session pessimistic concurrency demo (`SELECT ... FOR UPDATE` prevents negative stock).
- Deadlock detection handling (catches MySQL error 1213 and retries in DAO).

> **Report Sections Assigned:** Section 2 (Backend Solution), Section 6 (Backend Architecture), Section 13 (Problems & Solutions Log).

Member 2: Inventory, Supply Chain & Stock Flow

Theme: Branch inventory management, purchase orders, and stock reordering

Database Tables Owned:

`inventory`, `supplier`, `purchase_order`, `purchase_order_item`, `low_stock_alert`

Database Programming Objects:

- **Procedures:** `sp_add_stock` (ON DUPLICATE KEY UPDATE), `sp_receive_purchase` (marks received & increments stock per line), `sp_transfer_stock` (atomic inter-branch transfer).
- **Trigger:** `trg_inventory_low_stock` (AFTER UPDATE, creates alert below reorder level).
- **Function:** `fn_stock_level(branch_id, product_id)`.
- **View & Event:** `v_low_stock`, `ev_weekly_low_stock_scan`.

Application Implementation:

- **Spring Boot:** `lk.supermarket.inventory` (InventoryController, PurchaseOrderController, InventoryDao, SupplierDao).
- **Angular 17+:** `features/inventory/` (Stock table per branch, restock modal, transfer dialog, PO receiver).

Extra Features for Top Marks:

- Optimistic locking with a `version` column on inventory to prevent lost updates.
- Inter-branch stock transfer procedure with rollback validation.

> **Report Sections Assigned:** Section 7 (Tools & Tech), Section 8 (Hosting Justification), Section 15 (NoSQL Replacement).

Member 3: Products, Customers & Promotional Rules

Theme: Catalog management, customer loyalty tiers, and dynamic discount calculations

Database Tables Owned:

`product`, `category`, `customer`, `discount_rule`, `coupon`, `holiday`, `order_discount`

Database Programming Objects:

- **Procedures:** `sp_save_product`, `sp_register_customer`, `sp_apply_discounts` (Cursor evaluates Welcome, Holiday, Loyalty & Coupon with 50% max cap).
- **Triggers:** `trg_product_price_audit` (logs price alterations with USER()), `trg_rule_check` (bounds validation).
- **Functions:** `fn_customer_tier` (Bronze/Silver/Gold), `fn_is_holiday`, `fn_coupon_valid`.
- **View & Event:** `v_active_discounts`, `ev_expire_discount_rules`.

Application Implementation:

- **Spring Boot:** `lk.supermarket.product`, `customer`, `discount` (ProductDao, CustomerDao, DiscountDao).
- **Angular 17+:** `features/products/`, `customers/`, `discounts/` (Catalog CRUD, tier view, coupon generator, holiday calendar).

Extra Features for Top Marks:

- MySQL `FULLTEXT` index benchmarked against `LIKE '%x%'` using EXPLAIN.
- Dynamic stacking discount cursor engine with safety cap.

> **Report Sections Assigned:** Section 1 (Introduction), Section 3 (ER Diagram), Section 4 (Relational Mapping & 3NF), Section 5 (Table Structure).

Member 4: Reporting, Query Optimization & Big Data

Theme: Analytical aggregations, query execution plans, and large dataset handling

Database Tables Owned:

`daily_sales_summary`, `stock_audit_log` (Partitioned by Year)

Database Programming Objects:

- **Procedures:** `sp_monthly_report` (`GROUP BY ... WITH ROLLUP`), `sp_category_sales_cte` (Hierarchical sales via recursive CTE).
- **Trigger:** `trg_daily_sales_log` (audit logging on analytics data).
- **Function:** `fn_avg_daily_sales(branch_id, days)`.
- **Views & Events:** `v_daily_sales`, `v_product_ranking` (`RANK() OVER`), `v_discount_impact`, `ev_daily_sales_summary`, `ev_monthly_log_purge`.

Application Implementation:

- **Data Engineering:** Python bulk generator (`generate_orders.py`) creating 50,000+ realistic orders across 2 years.
- **Spring Boot:** `lk.supermarket.report` (ReportController, ReportService, ReportDao).
- **Angular 17+:** `features/reports/` (Chart.js visual dashboard, rollup summaries, revenue leaderboard).

Extra Features for Top Marks:

- `EXPLAIN ANALYZE` before-and-after query execution proof on 50k rows.
- RANGE Table Partitioning on `stock_audit_log` with partition pruning.

> **Report Sections Assigned:** Section 12 (Code Snippets), Section 14 (Cloud Architecture & Scaling).

Member 5: Security, RBAC, Data Masking & Administration

Theme: Enterprise database security, access control, audit defense, and disaster recovery

Database Tables Owned:

`employee`, `branch`, `employee_audit_log`, `security_event_log`

Database Programming Objects:

- **Procedures:** `sp_get_login`, `sp_deactivate_employee` (transactional with self-lockout check), `sp_change_user_role`.
- **Triggers:** `trg_employee_audit` (AFTER UPDATE, tracks role & status changes), `trg_security_alert` (prevents tampering with admin).
- **Functions:** `fn_mask_phone` (Dynamic Data Masking), `fn_check_password_strength`.
- **Views & Security:** `v_masked_customer_directory`, `v_security_audit_log`, MySQL 8 Roles (`cashier_role`, `manager_role`, `admin_role`).

Application & Platform Implementation:

- **Platform / DevOps:** Docker Compose orchestration (`docker-compose.yml`), MySQL users (`app_user`, `report_user`, `etl_user`).
- **Spring Boot:** Security plumbing (`SecurityConfig`, JWT filters, DataSourceConfig), `lk.supermarket.auth`, `audit`.
- **Angular 17+:** `core/` (JWT interceptor, Route guards), `features/admin/` (Employee admin, security log viewer).

Extra Features for Top Marks:

- MySQL 8 Role demo (`demo_cashier` allowed vs denied permissions matrix).
- Dynamic Data Masking view and `mysqldump` Point-in-Time Recovery.

> **Report Sections Assigned:** Section 9 (Database Security & RBAC), Section 10 (Least Privilege Accounts), Section 11 (Connection Code & Pool Tuning).

3. Academic Report Distribution (15 Sections, 3 per Member)

Sec #	Report Section Title & Scope	Assigned Author
1	Introduction & Problem Domain Description	Member 3 (Catalog & Customer)
2	Backend Solution & Architectural Overview	Member 1 (Sales & POS)
3	Entity Relationship (ER) Diagram & Cardinality Justification	Member 3 (Catalog & Customer)
4	Relational Mapping & Normalization to 3NF (Documenting Deliberate Denormalization)	Member 3 (Catalog & Customer)
5	Table Structure, Primary Keys, Composite Clustering & Constraints Design	Member 3 (Catalog & Customer)
6	Backend Architecture, DAO Pattern & Stored Procedure Execution Flow	Member 1 (Sales & POS)
7	Tools, Technologies & Software Framework Evaluation	Member 2 (Inventory & Supply)
8	Hosting & Cloud Infrastructure Justification (Localhost vs AWS RDS / Azure MySQL)	Member 2 (Inventory & Supply)
9	Database Security Measures, Access Control & RBAC Implementation	Member 5 (Security & Admin)
10	Database Accounts, Credentials Management & Principle of Least Privilege	Member 5 (Security & Admin)
11	Database Connection Management, HikariCP Connection Pooling & SSL Handshake	Member 5 (Security & Admin)
12	Advanced SQL Code Snippets Compilation (Procedures, Triggers, Views, Events, Cursors)	Member 4 (Analytics & Data)
13	Problems Encountered & Practical Solutions Log (Consolidated Team Issue Log)	Member 1 (Sales & POS)
14	Cloud Migration Strategy, Scaling, Read Replicas & Parameter Group Configuration	Member 4 (Analytics & Data)
15	NoSQL Replacement Evaluation & Polyglot Persistence Architecture (Redis / Mongo)	Member 2 (Inventory & Supply)

4. What This Work Distribution Achieves

⚖️ **Equitable SQL Footprint**

🎓 **Viva Defense Readiness**

🚀 **Zero Code Redundancy**

Every member has at least 1–2 stored procedures with transaction logic, 1–2 triggers, 1 UDF, 1 view, and specific indexes. Nobody is left with only frontend or non-database configuration tasks.

Each member has a dedicated, prepared 3-minute live demonstration script in MySQL Workbench proving their database objects function under stress and concurrency.

Every table and SQL object connects cleanly through Spring Boot JDBC endpoints into Angular Material screens, ensuring the application feels complete and cohesive.